POLYTECH DIJON

Introduction à l'IA : optimisation bio-inspirée

TP 2 : Initiation à l'algorithmique génétique

Objectif du TP

Ce TP a pour objectif de mettre en uvre les principaux mécanismes d'un algorithme génétique de base. À partir d'un framework Java fourni, vous étudierez la représentation des solutions et le déroulement de l'algorithme, puis vous implémenterez deux opérateurs classiques : une sélection par roulette biaisée et un croisement à deux points. Vous analyserez enfin leur influence sur le comportement de l'algorithme.

Compétences à valider à l'issue du TP

- ☐ Identifier les éléments constitutifs d'un algorithme génétique : chromosome, population, fonction objectif et opérateurs génétiques.
- ☐ Implémenter un opérateur de sélection probabiliste fondé sur la qualité des solutions.
- ☐ Implémenter et valider un opérateur de croisement à deux points.
- ☐ Configurer une expérimentation et interpréter le comportement d'un algorithme génétique.

Problème étudié

On souhaite utiliser un algorithme génétique afin de déterminer le plus grand nombre décimal pouvant être représenté sur n bits. Ce problème, volontairement simple, présente avant tout un intérêt pédagogique : sa solution optimale peut être déterminée directement, sans recourir à une méthode d'optimisation.

L'objectif est ici d'observer la manière dont un algorithme génétique explore progressivement l'espace de recherche et favorise, au fil des générations, les caractéristiques associées aux solutions les plus performantes.

Dans le problème considéré, les individus présentant des bits de poids fort positionnés à 1 possèdent généralement une meilleure valeur de fitness. Sous l'effet des mécanismes de sélection, de croisement et de mutation, ces caractéristiques devraient ainsi être progressivement favorisées au sein de la population, jusqu'à permettre la convergence vers la solution optimale (sans nécessairement l'atteindre).

Exercice 1 — Représentation des solutions et fonction objectif

Avant d'exécuter l'algorithme, analysez le problème étudié et répondez aux questions suivantes.

1. Identifiez les variables de décision du problème. À quoi correspond un gène ? Quelles valeurs peut-il prendre ?
2. Décrivez la représentation d'une solution. À quoi correspond le chromosome dans ce problème ?
3. Déterminez la taille de l'espace de recherche en fonction de n, puis donnez sa valeur pour $n = 16$.
4. Définissez la fonction objectif (vous écrirez la formule mathématique permettant de la calculer). S'agit-il d'un problème de minimisation ou de maximisation ?
5. Sans exécuter le programme, déterminez la solution optimale et la valeur optimale de la fonction objectif pour $n = 16$.

© Wahabou ABDOU — wahabou.abdou@ube.fr — Polytech Dijon 2026–2027

Exercice 2 — Prise en main du framework

Le projet Java fourni constitue un squelette d'algorithme génétique. Les classes sont organisées en plusieurs paquetages afin de séparer le problème, la représentation des solutions et les différents opérateurs.

1. Importez le projet dans votre environnement de développement, puis exécutez la classe `Main`. Vérifiez que le programme génère une population, fait évoluer celle-ci et affiche la meilleure solution rencontrée.

2. Repérez dans le code les classes assurant les fonctions suivantes :

   — définition et évaluation du problème ;

   — représentation d'un gène et d'une solution ;

   — sélection des parents ;

   — croisement ;

   — mutation ;

   — déroulement général de l'algorithme génétique.

3. Dans la classe `Main`, identifiez les paramètres suivants et précisez leur rôle :

   — `taillePopulation` ;

   — `nbGenerations` ;

   — `probaCroisement` ;

   — `probaMutation` ;

   — `graine`.

4. Le framework permet de choisir les opérateurs sans modifier le cur de l'algorithme. Repérez les deux instructions suivantes dans `Main` :

```
1 TypeSelection selection = TypeSelection.ALEATOIRE;
2 TypeCroisement croisement = TypeCroisement.UN_POINT;
```

   Expliquez le rôle de ces deux paramètres de configuration.

5. Exécutez plusieurs fois le programme avec une même valeur de `graine`, puis avec des valeurs différentes. Que constatez-vous ? Quel est l'intérêt de fixer une graine lors d'une expérimentation ?

Exercice 3 — Implémentation d'une sélection par roulette biaisée

La version initiale du framework utilise une sélection aléatoire uniforme : chaque individu possède la même probabilité d'être choisi comme parent, indépendamment de la valeur de sa fonction objectif.

La sélection par *roulette biaisée* attribue au contraire à chaque solution une probabilité de sélection dépendant de sa qualité. Pour un problème de maximisation et des valeurs de fitness non négatives,

© Wahabou ABDOU — wahabou.abdou@ube.fr — Polytech Dijon 2026–2027

le principe peut être exprimé par :

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j},$$

où N désigne la taille de la population.

1. Étudiez la classe `SelectionAleatoire`. Identifiez la méthode que doit redéfinir tout nouvel opérateur de sélection.
2. Complétez la méthode `selectionner()` de la classe `SelectionRoulette`. Votre implémentation devra notamment :
   — associer un poids positif à chaque solution ;
   — calculer la somme des poids ;
   — générer une valeur aléatoire dans l'intervalle correspondant à la roulette ;
   — parcourir les poids cumulés afin d'identifier la solution sélectionnée.
3. Modifiez uniquement la configuration de la classe `Main` afin d'utiliser la roulette :

```
TypeSelection selection = TypeSelection.ROULETTE;
```

   Vérifiez le bon fonctionnement de votre implémentation.
4. Pour une petite population dont vous afficherez les valeurs de fitness, vérifiez qualitativement que les solutions de meilleure qualité sont sélectionnées plus fréquemment, sans pour autant rendre la sélection déterministe.

Exercice 4 — Implémentation d'un croisement à deux points

Le framework fournit un croisement à un point dans la classe `Croisement1Point`. Le croisement à deux points consiste à choisir deux positions du chromosome puis, lorsque le croisement est déclenché, à échanger entre les deux parents la portion comprise entre ces positions.

1. Étudiez l'implémentation de `Croisement1Point`. Identifiez notamment la création des enfants, le test de la probabilité de croisement et l'échange des gènes.
2. Complétez la méthode `croiser()` de la classe `Croisement2Points`. Les deux points de croisement devront être distincts et ordonnés avant l'échange des gènes.
3. Testez votre opérateur sur deux chromosomes connus. Pour faciliter la vérification, utilisez temporairement une probabilité de croisement égale à 1 et affichez les deux parents ainsi que les deux enfants.
4. Activez ensuite le nouvel opérateur dans `Main` :

```
TypeCroisement croisement = TypeCroisement.DEUX_POINTS;
```

   Rétablissez enfin la valeur initiale de la probabilité de croisement et vérifiez le fonctionnement global de l'algorithme.

© Wahabou ABDOU — wahabou.abdou@ube.fr — Polytech Dijon 2026–2027

Exercice 5 — Expérimentation et analyse

Une comparaison pertinente des opérateurs nécessite de contrôler les autres paramètres de l'algorithme. Pour une même configuration, utilisez donc les mêmes valeurs de `taillePopulation`, `nbGenerations`, `probaCroisement`, `probaMutation` et `graine`.

1. Comparez successivement les configurations suivantes :
   - sélection aléatoire + croisement à un point ;
   - roulette biaisée + croisement à un point ;
   - roulette biaisée + croisement à deux points.
2. Pour chaque configuration, relevez la meilleure valeur obtenue. Répétez l'expérience avec plusieurs graines afin de ne pas tirer de conclusion à partir d'une seule exécution.
3. Analysez vos résultats. Discutez en particulier l'effet de la pression de sélection et de la modification de l'opérateur de croisement sur la recherche de bonnes solutions.

© Wahabou ABDOU — wahabou.abdou@ube.fr — Polytech Dijon 2026–2027